Raspberry Pi 5 Ubuntu 20.04 support

Share

Please check the documentation carefully to see if the Raspberry Pi 5 supports Ubuntu 20.04 LTS.

Searched the web, read 2 pages

No, the Raspberry Pi 5 does not support Ubuntu 20.04 LTS. Canonical's official Raspberry Pi documentation (last updated July 10, 2026) shows the Pi 5 as supported only on 24.04 LTS and 26.04 LTS.

- **Support matrix:** The Pi 5 row is blank under 22.04 LTS. It is marked as server-supported, desktop-supported and certified on both 24.04 LTS and 26.04 LTS. The page notes that only releases in standard support are included, and support for prior LTS releases may still be available via Ubuntu Pro. 20.04 no longer appears in the matrix at all. ubuntu
- **When Pi 5 support began:** Ubuntu 24.04 LTS was the first LTS release supporting the Raspberry Pi 5, with arm64 server and desktop images. Pi 5 support first appeared in 23.10, which one reviewer said was not sufficient, with many features not working. gamingonlinux impress
- **No backport:** A Canonical engineer said the company has no plans to backport Pi 5 support to 22.04 LTS. That makes 20.04, which is older still, even less likely. slashgear
- **The 20.04 images:** The Ubuntu 20.04.5 images on that page are listed for older boards. Ubuntu 20.04 was available for the Raspberry Pi versions 2, 3, and 4, and its certification dates from before the Pi 5 existed. ionos

I can't confirm from the docs that a 20.04 image won't boot at all on a Pi 5. The docs only show it isn't a supported combination, and nothing there suggests the Pi 5 hardware is handled by the 20.04 kernel and firmware.

What to use instead:

- Use Ubuntu 24.04 LTS or 26.04 LTS, both with arm64 images.
- If you need 20.04 for software compatibility, run it in a container or VM on top of 24.04 rather than on the bare hardware.
- If the Pi 5 has an older bootloader, check the EEPROM date with `rpi-eeprom-update`. The docs list a minimum date of 2025-02-11 for the Pi 5 on 26.04 and no minimum for 24.04.

Sources:

- Install Ubuntu on the Raspberry Pi (Canonical)
- Ubuntu 24.04 LTS for Raspberry Pi 5 (OMG! Ubuntu)
- Can Raspberry Pi 5 Run Ubuntu? (SlashGear)

- Ubuntu 24.04 LTS is now available (GamingOnLinux)
- Raspberry Pi: Ubuntu installation and setup (IONOS)
- Ubuntu日和 第52回 (PC Watch)

```
board: rev d04170 0014 serial ebad2f57 2c:cf:67:4f:b6:7d
boot: mode USB-MSD 4 order f41 retry 0/128 restart 21/-1
SD: card detected 001b534d454432533530992367aea183
part: 1 mbr [0x0c:00000800 0x83:00080800 0x00:00000000 0x00:00000000]
power: supply: USB-PD 5000 mA CC1 PMIC: reset normal 0x0 usb_oc=0
net: up ip: 10.226.45.177 sn: 255.255.252.0 gw: 10.226.44.1
tftp: 0.0.0.0 00:00:00:00:00:00
display: DISPO: HPD=0 EDID=none #0 DISP1: HDMI HPD=1 EDID=ok #2
Failed to open partition 4
Trying partition: 5
Failed to open partition 5
Trying partition: 0
type: 32 1ba: 2048 'mkfs.fat'
• system-boot' clusters 516190 (1)
Trying partition: 1
type: 32 1ba: 2048 'mkfs.fat'" system-boot'
clusters 516190 (1)
Read config. txt bytes
1914 hnd 0x9ea8
Read syscfg. txt bytes
327 hnd 0x5e99
Read usercfg. txt bytes
200 hnd 0x5e3a
usb_max_current_enable default 0 max-current 5000
Device-tree file "bcm2712-rpi-5-b.dtb" not found.
The installed operating system (OS) does not indicate support for Raspberry Pi 5
Update the OS or set os_check=0 in config.txt to skip this check.
Boot mode: USB-MSD (04) order f
```

Checking the Pi 5 board revision and matching the OS image

That log matches what the docs predicted. This is a Pi 5 (board rev d04170), and it's booting from the USB drive, but the image on it has no Pi 5 support.

- `Device-tree file "bcm2712-rpi-5-b.dtb" not found`: This is the Pi 5's device tree file, and the boot partition (`system-boot`) doesn't contain it. The image's firmware and kernel files predate the Pi 5.
- `The installed operating system (OS) does not indicate support for Raspberry Pi 5`: The bootloader found the same problem and stopped. This is a compatibility check, not a hardware fault. Your power, network and HDMI all look fine (USB-PD 5000 mA, link up, display detected).

The log doesn't name the OS version, so I'm inferring that this is the Ubuntu 20.04 image from your earlier question. It's very likely, since 20.04 has no Pi 5 support.

The fix is to reflash with a supported release:

1. Open Raspberry Pi Imager on another computer and choose Raspberry Pi 5 as the device.
2. Pick Ubuntu Server 24.04 LTS (64-bit), or 26.04 LTS. The desktop edition is also available if you have 4 GB of RAM or more.
3. Write it to the same USB drive (or SD card) and boot again.

Don't use `os_check=0`. The log suggests it, but it only skips the check. The files the Pi 5 needs are still missing, so it would fail further along. Reflashing is the real fix.

If you need 20.04 for a particular piece of software, run it in a container or VM on top of 24.04.

What if I need to run Ubuntu 20.04 on a Raspberry Pi 5? What should I do?

Listing containers, VMs and chroot options for Ubuntu 20.04 ▸

You can't run 20.04 directly on the Pi 5's hardware, but you can run a 20.04 environment on top of a supported OS. Which method fits depends on why you need 20.04.

1. Container (easiest, if you only need the 20.04 userland)
Install Ubuntu 24.04 on the Pi, then run 20.04 in Docker or LXD. Canonical publishes arm64 images, so it runs natively with no emulation:

```
sudo apt install docker.io
sudo docker run -it --rm ubuntu:20.04 bash
```

This gives you 20.04's packages, libraries and toolchain. It shares the host's 24.04 kernel, so it won't help if you need the 20.04 kernel itself, or kernel modules built for it.

2. Virtual machine (if you need a full 20.04 system with its own kernel)
Use QEMU/KVM on the 24.04 host and install the 20.04 arm64 server image in a VM. You get a separate kernel and a fully isolated system. The cost is some performance overhead and more setup than a container. The VM's hardware access (GPIO, camera, and so on) is limited unless you pass devices through, which is fiddly.

3. chroot with debootstrap (lightweight middle ground)
You can build a 20.04 root filesystem inside the host with `debootstrap focal` and chroot into it. It's like a container with less isolation and less tooling.

4. Bare metal: not realistic
I wouldn't try mixing a Pi 5 kernel and firmware from a newer image into a 20.04 install. It

would be unsupported, hard to maintain, and Canonical has said it won't backport Pi 5 support to older releases.

Security. 20.04's standard support period has ended, so anything you run in it will stop getting regular updates. Extended Security Maintenance through Ubuntu Pro extends security patches for it. I haven't checked the current Pro terms and dates, so confirm them on Canonical's site before relying on them. Pro covers package security updates only, not Pi 5 hardware support.

If you tell me what depends on 20.04 (an old ROS version, a vendor SDK, a specific Python or library version, a kernel module), I can say which option fits and walk through the setup.

If I have to choose "4. Bare metal: not realistic," what should I do?

Noting which GPU and camera stacks break on the old userland ▸

If you do go bare metal, the only workable approach is a hybrid: the Pi 5's supported kernel and boot files from 24.04, with a 20.04 root filesystem on top. 20.04's own kernel can't boot a Pi 5, so you're really running a 20.04 userland on a newer kernel. This is my general knowledge rather than something from the docs I searched, and I haven't tested it, so treat it as a starting point.

Rough procedure

1. **Get a working 24.04 base.** Flash Ubuntu Server 24.04 arm64, boot it, update it, and run `sudo rpi-eeprom-update -a` so the bootloader is current. This image is your kernel donor.
2. **Build a 20.04 root filesystem.** Using a second drive, partition it and run `debootstrap --arch=arm64 focal` against the ports mirror (`ports.ubuntu.com/ubuntu-ports`). Chroot in and set up a user, hostname, SSH, networking (netplan works on focal) and `/etc/fstab`.
3. **Keep the 24.04 boot partition.** It holds the Pi 5 kernel, device tree (`bcm2712-rpi-5-b.dtb`), initrd, `config.txt` and `cmdline.txt`. Point `root=` in `cmdline.txt` at your new root partition.
4. **Copy the kernel pieces into the 20.04 root.** Copy `/lib/modules/<24.04 kernel version>` and `/lib/firmware` from the 24.04 install, because the 20.04 packages don't ship matching ones.
5. **Boot and test.** Since the boot files come from the 24.04 image, the "does not indicate support for Raspberry Pi 5" error should go away.

Maintenance problems

- **Kernel updates.** 20.04's `linux-raspi` package is an old kernel with no Pi 5 support, so don't let apt install or switch kernels. Hold it with `apt-mark hold`. Kernel updates are then manual, by copying boot files and modules from a 24.04 install again.

- **Security updates.** 20.04's standard support has ended, so patches need Ubuntu Pro (ESM). Check Canonical's current terms.
- **Nothing here is supported.** No one will help you debug it, and a bad update can leave it unbootable.

What will likely not work well

- **GPU and desktop.** 20.04's Mesa is far too old for the Pi 5's graphics, so expect no acceleration. Headless server use is much safer.
- **Camera.** The libcamera stack is the same limitation, and the docs say it's not operational before Ubuntu 25.04 anyway.
- **GPIO libraries.** The Pi 5 moved I/O onto the RP1 chip, so older libraries like RPi.GPIO and wiringPi don't work. Use libgpiod or lgpio, and expect to do this on any OS.

A shortcut. If the 20.04 userland doesn't need its own PID 1, `systemd-nspawn --boot` on a 24.04 host gives you nearly the same result with far less risk. You only need the hybrid if 20.04 must boot as the whole system.

If you tell me what requires 20.04 (a vendor SDK, an old ROS release, specific libraries), I can tell you whether the container route covers it, or write out the exact commands for the hybrid.

> Can I achieve support for the Raspberry Pi 5 by flashing the image `ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz.zsync` onto an SD card and then editing the image file?

Checking whether Pi 5 needs more than config edits ▸

Technically possible, but not by editing settings. The image has to be modified to carry a Pi 5 kernel and boot files. That's the hybrid approach from my last message, done on the image file instead of a running system. I haven't tested this.

First, the file you named. `.img.xz.zsync` isn't the image. It's a small metadata file that `zsync` uses to download or resume the real image. You want `ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz`, from the same folder on cdimage.ubuntu.com (the docs page I read links it).

Why editing the 20.04 image alone won't work

- The boot log you posted shows the missing `bcm2712-rpi-5-b.dtb`. Dropping in that one file wouldn't fix it. The 20.04 kernel has to drive the Pi 5's SoC and its RP1 I/O chip, and as far as I know it predates that hardware by a wide margin. I haven't verified the exact kernel versions.
- `os_check=0` only skips the compatibility check. The kernel still can't run the board.

What you'd have to replace in the image

- **Boot partition (`system-boot`):** the kernel, initrd, `bcm2712-rpi-5-b.dtb`, overlays, firmware files, `config.txt`, and `cmdline.txt`. All of these must come from a 24.04 Pi image.
- **Root partition:** the matching kernel modules (`/lib/modules/<version>`) and `/lib/firmware`.

Procedure

1. Decompress both the 20.04.5 and 24.04 images (`xz -d`).
2. Attach each as a loop device: `sudo losetup -fP --show <image>`. Then mount partition 1 (boot) and partition 2 (root) of each.
3. Copy everything from the 24.04 boot partition into the 20.04 boot partition, replacing what's there.
4. Copy the 24.04 `/lib/modules/*` and `/lib/firmware` into the 20.04 root.
5. Check `cmdline.txt` still points at the 20.04 root (`root=LABEL=writable`), and that the labels and `/etc/fstab` match.
6. Unmount, detach, and flash the edited image to the SD card.

It's easier to start from the 24.04 image and swap in the 20.04 root filesystem, since the boot partition is already correct. Either way you end up with the same hybrid.

Caveats

- The Pi 5 will boot a 24.04 kernel under a 20.04 userland. It's unsupported and untested by Canonical.
- Don't let apt replace or update the kernel in the 20.04 root. Hold the kernel packages, or a routine update can leave the system unbootable.
- The security and hardware limits from my last message still apply.

If you only need 20.04's software, a container or `systemd-nspawn` is much simpler. If you want to go ahead with the image edit, I can write the exact commands, including the `fstab` and `cmdline.txt` checks.

> I've decided to start with the 24.04 image and then replace the root filesystem with the one from 20.04. How do I go about doing this?

Stashing the modules and firmware before swapping the root ▸

This approach is simpler than editing the 20.04 image, because the 24.04 image already has a Pi 5 boot partition and kernel. You keep its boot partition and kernel modules and swap

everything else in the root partition for the 20.04 filesystem. I haven't tested this exact procedure, so keep an untouched copy of the 24.04 image as a fallback.

You'll need a Linux machine (x86 is fine, since this is only file copying) with `rsync` and `xz-utils`, and about 10 GB free.

1. Download and unpack both images

```
mkdir pi5-focal && cd pi5-focal
wget https://cdimage.ubuntu.com/releases/noble/release/ubuntu-24.04.5-preinstalled-server-arm64+raspi.img.xz
wget https://cdimage.ubuntu.com/releases/focal/release/ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz
xz -dk *.xz
mv ubuntu-24.04.5-preinstalled-server-arm64+raspi.img noble.img
mv ubuntu-20.04.5-preinstalled-server-arm64+raspi.img focal.img
cp noble.img noble-pristine.img
```

2. Mount them

Partition 1 is the FAT boot partition (`system-boot`) and partition 2 is the ext4 root (`writable`).

```
NOBLE=$(sudo losetup -fP --show noble.img)
FOCAL=$(sudo losetup -fP --show focal.img)
sudo mkdir -p /mnt/nboot /mnt/nroot /mnt/froot
sudo mount ${NOBLE}p1 /mnt/nboot
sudo mount ${NOBLE}p2 /mnt/nroot
sudo mount -o ro ${FOCAL}p2 /mnt/froot
```

Check the 20.04 filesystem fits in the 24.04 root partition:

```
df -h /mnt/nroot
sudo du -sxh /mnt/froot
```

If it's tight, unmount, enlarge first with `truncate -s +2G noble.img`, then `sudo growpart ${NOBLE} 2` and `sudo resize2fs ${NOBLE}p2` (growpart is in `cloud-guest-utils`).

3. Stash the 24.04 kernel modules and firmware

```
sudo mkdir -p /tmp/stash
sudo rsync -aHAX /mnt/nroot/lib/modules/  /tmp/stash/modules/
sudo rsync -aHAX /mnt/nroot/lib/firmware/ /tmp/stash/firmware/
```

4. Replace the root contents with 20.04

```
sudo rsync -aHAX --numeric-ids --delete --exclude=/lost+found /mnt/froot/
/mnt/nroot/
```

5. Put the 24.04 modules and firmware back

```
sudo mkdir -p /mnt/nroot/lib/modules /mnt/nroot/lib/firmware
sudo rsync -aHAX /tmp/stash/modules/  /mnt/nroot/lib/modules/
sudo rsync -aHAX /tmp/stash/firmware/ /mnt/nroot/lib/firmware/
```

The firmware is merged over 20.04's older files, not replacing them.

6. Verify before flashing

```
head -2 /mnt/nroot/etc/os-release                          # should say 20.04
ls /mnt/nroot/lib/modules                                  # should include a
6.8.x-raspi directory
cat /mnt/nroot/etc/fstab                                   # LABEL=writable on /,
LABEL=system-boot on /boot/firmware
grep -o 'root=[^ ]*' /mnt/nboot/cmdline.txt                # root=LABEL=writable
ls /mnt/nboot | grep -E 'vmlinuz|initrd|bcm2712-rpi-5-b.dtb'
```

The last command should list all three, including the `bcm2712-rpi-5-b.dtb` your boot log said was missing. If the fstab uses labels other than `writable` and `system-boot`, edit it to match.

7. Clean up and flash

```
sudo umount /mnt/nboot /mnt/nroot /mnt/froot
sudo losetup -d $NOBLE $FOCAL
```

Write `noble.img` (now modified) to your SD card or USB drive with Raspberry Pi Imager ("Use custom") or `dd`. On first boot, cloud-init uses the 24.04 boot partition's `user-data`, so the login is `ubuntu` / `ubuntu` with a forced password change. It should also expand the root partition to fill the media.

8. Immediately after first boot, hold the kernel packages

This is the step that matters most for keeping the system bootable. The 20.04 root still has its own `linux-raspi` kernel packages. If apt updates them, their hooks copy a 20.04 kernel over the Pi 5 boot files and the board stops booting. Run this first:

```
dpkg-query -W -f='${Package}\n' 'linux-*raspi*' 'linux-firmware' 'flash-kernel'
| sudo xargs apt-mark hold
```

Then check that `uname -r` shows 6.8.x and `/etc/os-release` shows 20.04.

Known issues

- **Pi 5 kernel on a 20.04 userland.** Basic server use should work. Anything tied to the old userland's graphics or camera stack won't.
- **Kernel updates are manual.** To update, repeat the 24.04 boot partition and modules copy.
- **Debugging.** If it doesn't boot, a USB-UART debug probe on the Pi 5's UART header (the 3-pin header between the micro-HDMI ports) will show kernel messages far better than HDMI will.
- **Support.** 20.04 is past standard support, and this combination is unsupported by Canonical.

I can turn these steps into a single script with error checks if you'd like.

> I don't have a Linux computer; I used Raspberry Pi Imager on Windows to install Ubuntu 24.04 onto an SD card.

Comparing a Pi-only route with WSL2 image editing ▸

That's fine. You can run the same procedure on Windows with **WSL2** (Windows Subsystem for Linux), working on the image files and then flashing the result with Raspberry Pi Imager. It needs Windows 10 (version 2004 or later) or Windows 11. I haven't tested this on your setup.

1. Install WSL2

In PowerShell opened as Administrator:

```
wsl --install -d Ubuntu-24.04
```

Reboot if asked, then open "Ubuntu 24.04" from the Start menu and create a user. Inside it, install the tools:

```
sudo apt update && sudo apt install -y rsync xz-utils wget cloud-guest-utils
```

2. Do the work inside WSL's own filesystem

Work in your Linux home folder (`~`), not under `/mnt/c`. The Windows drives don't preserve Linux permissions and ownership, which would break the root filesystem. You need roughly 12 GB free on your Windows drive.

Then follow my previous steps 1-6 unchanged, with these differences:

- **Resizing.** If the 20.04 filesystem doesn't fit, resize the image file *before* attaching it as a loop device:

```
truncate -s +2G noble.img
sudo growpart noble.img 2
```

Then attach it with `losetup`, and run `sudo e2fsck -f ${NOBLE}p2 && sudo resize2fs ${NOBLE}p2` before mounting.

- **Loop devices.** WSL2 normally supports `losetup -P` and mounting ext4/vfat. If `losetup` fails with an error about finding a loop device, tell me the exact message and I'll suggest a workaround.

3. Get the image back to Windows and flash it

After unmounting and detaching as in step 7, copy the file out:

```
cp ~/pi5-focal/noble.img /mnt/c/Users/<your-windows-username>/Desktop/
```

In Raspberry Pi Imager, choose **Choose OS → Use custom**, select `noble.img`, pick your SD card, and write it. Imager's customisation options (hostname, user, SSH) may or may not apply to a custom image, so if you can't log in, use the default `ubuntu` / `ubuntu` login, which will force a password change.

Don't flash `noble-pristine.img`. That's your untouched backup if something goes wrong.

4. After first boot

Do step 8 from before **immediately**: hold the kernel packages so apt can't replace the Pi 5 kernel. Then confirm with `uname -r` (should be 6.8.x) and `cat /etc/os-release` (should say 20.04).

Fallback: skip WSL and do it on the Pi

If WSL gives you trouble, you can do the equivalent on the Pi 5 itself, since your current 24.04 SD card already boots it. Plug in a spare USB drive or SSD, build a 20.04 filesystem on it with `debootstrap` (`sudo apt install debootstrap`, then `sudo debootstrap --arch=arm64 focal /mnt/usb http://ports.ubuntu.com/ubuntu-ports`), copy over the 24.04 `/lib/modules` and `/lib/firmware`, and change `root=` in `/boot/firmware/cmdline.txt` to the USB drive. The SD card keeps the boot partition and the USB drive becomes the 20.04 system. It's a little more manual (users, fstab and

networking need setting up in a chroot), but it needs no extra computer and the 24.04 SD card stays as an easy way to get back in.

Do you want me to write out the WSL path as one script with checks, or the full steps for the Pi-only fallback?

